

2. Write a C program to simulate a multi-level queue scheduling algorithm considering the following scenario. All the processes in the system are divided into two categories – system processes and user processes. System processes are to be given higher priority than user processes. Use FCFS scheduling for the processes in each queue.

Code:

```
#include <stdio.h>

int main() {
    int n;
    printf("Enter number of processes: ");
    scanf("%d", &n);

    int pid[n], at[n], bt[n], rt[n], ct[n], wt[n], tat[n], type[n];

    for (int i = 0; i < n; i++) {
        printf("\nProcess %d\n", i+1);
        pid[i] = i+1;

        printf("Enter arrival time: ");
        scanf("%d", &at[i]);

        printf("Enter burst time: ");
        scanf("%d", &bt[i]);

        printf("Enter type (0 = System, 1 = User): ");
        scanf("%d", &type[i]);

        rt[i] = bt[i];
    }
    for (int i = 0; i < n - 1; i++) {
        for (int j = 0; j < n - i - 1; j++) {
            if (at[j] > at[j + 1]) {
                int temp;
```

```

        temp = at[j];
        at[j] = at[j + 1];
        at[j + 1] = temp;
        temp = bt[j];
        bt[j] = bt[j + 1];
        bt[j + 1] = temp;
        temp = rt[j];
        rt[j] = rt[j + 1];
        rt[j + 1] = temp;
        temp = pid[j];
        pid[j] = pid[j + 1];
        pid[j + 1] = temp;
        temp = type[j];
        type[j] = type[j + 1];
        type[j + 1] = temp;
    }
}
}
int systemQ[100], userQ[100];
int sf = 0, sr = 0, uf = 0, ur = 0;
int current_time = 0, completed = 0, i = 0;
int current = -1;
int gantt[1000], gindex = 0;
while (completed < n) {
    while (i < n && at[i] <= current_time) {
        if (type[i] == 0)
            systemQ[sr++] = i;
        else
            userQ[ur++] = i;
        i++;
    }
    if (current != -1) {
        if (type[current] == 1 && sf < sr) {
            userQ[ur++] = current;
            current = -1;
        }
    }
    if (current == -1) {
        if (sf < sr)
            current = systemQ[sf++];
    }
}

```

```

        else if (uf < ur)
            current = userQ[uf++];
        else {
            gantt[gindex++] = -1;
            current_time++;
            continue;
        }
    }
    gantt[gindex++] = pid[current];
    rt[current]--;
    current_time++;
    if (rt[current] == 0) {
        ct[current] = current_time;
        completed++;
        current = -1;
    }
}

float total_wt = 0, total_tat = 0;
for (int i = 0; i < n; i++) {
    tat[i] = ct[i] - at[i];
    wt[i] = tat[i] - bt[i];

    total_wt += wt[i];
    total_tat += tat[i];
}

printf("\nID\tType\tAT\tBT\tCT\tWT\tTAT\n");
for (int i = 0; i < n; i++) {
    if (type[i] == 0)
        printf("%d\tSystem\t", pid[i]);
    else
        printf("%d\tUser\t", pid[i]);

    printf("%d\t%d\t%d\t%d\t%d\n", at[i], bt[i], ct[i], wt[i], tat[i]);
}

printf("\nAverage Waiting Time: %.2f", total_wt / n);
printf("\nAverage Turnaround Time: %.2f\n", total_tat / n);
printf("\nGantt Chart:\n| ");
for (int i = 0; i < gindex; i++) {
    if (gantt[i] == -1)
        printf("Idle | ");

```

```

    else
        printf("P%d | ", gantt[i]);
    }
    printf("\n0 ");
    for (int i = 1; i <= gindex; i++) {
        printf(" %d ", i);
    }
    printf("\n");
    return 0;
}

```

OUTPUT:

```

Enter number of processes: 3

Process 1
Enter arrival time: 1
Enter burst time: 8
Enter type (0 = System, 1 = User): 0

Process 2
Enter arrival time: 4
Enter burst time: 7
Enter type (0 = System, 1 = User): 1

Process 3
Enter arrival time: 2
Enter burst time: 3
Enter type (0 = System, 1 = User): 1

ID      Type  AT  BT  CT  WT  TAT
1      System  1   8   9   0   8
3       User   2   3  12   7  10
2       User   4   7  19   8  15

Average Waiting Time: 5.00
Average Turnaround Time: 11.00

Gantt Chart:
| Idle | P1 | P1 | P1 | P1 | P1 | P1 | P1 | P1 | P3 | P3 | P3 | P2 | P2 | P2 | P2 | P2 | P2 |
0  1  2  3  4  5  6  7  8  9 10 11 12 13 14 15 16 17 18 19

Process returned 0 (0x0)   execution time : 47.393 s
Press any key to continue.

```